

Automated task-dataset generation pipeline: progress

Automated task-dataset generation pipeline: progress

My hands-on log for the automated dataset + training module (Automated_Dataset_and_Reasoning_Training.pdf): leverage an LLM to turn a real, LINKED help-desk knowledge base into a task-specific fine-tuning dataset with reasoning traces, keep it balanced, mix in a little general reasoning + tool-calling so the model does not lose those abilities, then train one task and check it improved without regressing. Builds on the corpus-to-specialist deep dive (Tracks A and B under ../task_specific/), reusing the same spine. Generation runs LOCAL on Ollama; training runs on a free Kaggle T4. Small on purpose (a few hundred cases) to get the loop right, not scale.

Status: [] not started, [~] in progress, [x] done

Stage	What	Status
0	Understand the linked knowledge base	[x]
1	Pick the task + survey/confirm relevant fields	[x]
2	Build seed + sacred gold (split by issue id)	[x]
3	Generate reasoning traces, judge, filter, balance	[x]
4	Rehearsal mix (general reasoning + tool-calling)	[x]
5	Train one LoRA (control vs real)	[x]
6	Evaluate two axes + compare	[x]
7	Iterate / write up	[~]

Ran end to end. The reasoning-traced synthetic data plus the rehearsal mix beat the base on the task AND held the regression probes; the seed-only control regressed. Numbers below. Stage 7 is [~] because the acceptance criterion is already met (task up, no regression), so a second iteration is optional.

The corpus (verified)

A real, linked Jira-style help desk export (see FEATURES.md, EXAMPLE.md, db.png): - issues.csv: one row per issue, id is the unique key (66,691 issues). Metadata + aggregated workflow time-in-state (wf_/wfe_) + the resolution outcome. Issue-level resolution: Done 62,034, Won't Do 2,991, Duplicate 661, Cannot Reproduce 152, null 853. - issues_snapshot.csv: per-assignee turn snapshots (join on id). - issues_change_history.csv: raw assignee/status change log (issueid -> id). - sample_utterances.csv: masked conversation text, but only ~360 appraised issues have it (and only 6 of those are Won't Do). - issues_snapshot_sample.xlsx: human Q1/Q2/Q3 performance appraisal (reserved for a later text task).

Decisions

- Task: binary resolution, Done vs Won't Do, over the STRUCTURED linked context. macro-F1 is the headline metric (Done dominates). The label is real from the data, so

only the reasoning trace is synthetic (the grounding rule).

- Text is too sparse for resolution (6 Won't Do issues have utterances), so the conversation is NOT used here; serialize.py is built text-aware so an appraisal/text task is a cheap follow-on, not a rewrite.
- Leakage is the central correctness risk. The survey+confirm gate plus serializer excludes remove the final status, the resolution date, and outcome-coupled workflow states (rejected/cancelled/done) and terminal statuses in the handling path.

What is built (code complete)

- ☒ link.py joins the four tables by issue id; verified against EXAMPLE.md (issue 1004364: Done/closed, 2 turns 4hghq/4sii, 5 history events).
- ☒ serialize.py renders one issue's leakage-aware linked context (metadata + workflow time-in-state + pre-resolution handling path; text-aware flag, off for resolution).
- ☒ phase1_seed/survey.py (the configure step): local model proposes relevant vs leakage fields, writes data/field_survey.json; human confirms; downstream hard-gates on it.
- ☒ phase1_seed/build_seed.py: balanced gold + seed split by issue id; smoke-run offline.
- ☒ phase2_synthetic/gen_reasoning.py: per ticket, fix the verified label and generate a justifying reasoning trace, varied length; grounding guard drops mismatched answers.
- ☒ phase2_synthetic/judge.py: faithfulness gate (trace must follow from the fields).
- ☒ phase2_synthetic/filter.py: decontaminate by id + near-dedup + class/length balance.
- ☒ phase2_synthetic/mix_rehearsal.py: ~75/25 task : (GSM8K + tool-calling) rehearsal.
- ☒ phase3_train/train.py: QLoRA on Qwen2.5-0.5B-Instruct, assistant-only loss, max_len 1408.
- ☒ eval/evaluate.py + compare.py: macro-F1/acc on gold + sentinel/reasoning/tools probes.
- ☒ kaggle_notebook/track_c_kaggle.md: cloud training cells (generation stays local).

Run it: done, end to end

- ☒ survey.py -> reviewed data/field_survey.json, confirmed the leakage excludes.
- ☒ build_seed.py -> gold 200 (100 Done / 100 Won't Do), seed 16 (8/8).
- ☒ gen_reasoning.py -> judge.py -> filter.py -> train_synth.jsonl (222 rows).
- ☒ mix_rehearsal.py -> train_mix.jsonl (296 = 222 task + ~74 rehearsal, ~75/25).
- ☒ trained seed (control) and seed-synth (real); seed-synth final train loss ~1.08.
- ☒ evaluated base / seed / seed-synth and compared (data/comparison.json).

Two-axis result (gold = 200 rows, 100 Done / 100 Won't Do; probes higher is better)

condition	accuracy	macro-F1	sentinel	reasoning	tools
base	0.520	0.423	0.917	0.800	0.750
seed	0.510	0.355	0.833	0.800	0.750
seed-synth	0.630	0.627	0.917	1.000	0.875

Read: the reasoning-traced synthetic data plus the rehearsal mix lifted task macro-F1 from 0.423 to 0.627 (+0.204) while HOLDING or improving every regression probe (sentinel flat at 0.917, reasoning 0.80 -> 1.00, tools 0.75 -> 0.875, plausibly from the GSM8K + tool-calling rehearsal). The seed-only control went the other way, macro-F1 0.423 -> 0.355 and

sentinel 0.917 -> 0.833: plain fine-tuning on the tiny 16-row seed overfit and forgot. So the synthetic + replay run is the clear winner on both axes, exactly the acceptance criterion (task up, no regression). This is the headline before/after for the module.

Test the trained model

run_trained.py loads the adapter and predicts a ticket's resolution interactively (enter a ticket id, or blank for the next gold ticket), or one-shot with `-issue-id`. See README.

Risks / notes

- Confirm the survey leakage excludes before generating; this is where the task can quietly cheat.
- Class imbalance: keep gold balanced, cap Done in the training pool, read macro-F1.
- Same-model judge has self-preference/length bias; fine for a learning run, a stronger judge is the real fix.
- The 4 GB local GPU is for generation only; training goes to Kaggle (T4).

Run order, commands, and the per-file detail are in README.md.